Heterogeneous Expert PLE: An Explainable Multi-Task Architecture for Financial Product Recommendation

Author 1[1], Author 2[1], Author 3[1]

[1]*Independent Research*
(affiliated with a Korean public financial institution)

Abstract. Financial product recommendation requires not only predictive accuracy but also regulatory-compliant explainability. Existing multi-task learning approaches (MMoE, PLE) employ homogeneous MLP experts that suffer from *expert collapse* — all gates converge to a single expert, eliminating the diversity that justifies the MoE architecture — and lack structural interpretability. We propose *Heterogeneous Expert PLE*, a Progressive Layered Extraction architecture where seven **architecturally distinct** experts — DeepFM, Temporal Ensemble (Mamba+LNN+Transformer), Hyperbolic GCN, PersLay, Causal (NOTEARS), LightGCN, and Optimal Transport — share a common basket, providing a **structural guarantee** against expert collapse: experts with fundamentally different inductive biases cannot converge to the same function. CGC gates learn task-specific expert compositions whose weights are inherently interpretable as business-meaningful explanations — "35% spending trend (Temporal) + 28% product hierarchy (HGCN)" — without post-hoc SHAP/LIME. Ablation on a 1M-customer benchmark reveals that different experts specialize in different task types: LightGCN and Causal experts dominate multiclass tasks (Macro F1 +0.20), while Causal and TDA experts excel at regression tasks (MAE –0.024), validating the heterogeneous design. Combined with Adaptive Task Transfer (adaTT) over four financial-DNA task groups and multi-disciplinary feature engineering spanning eleven academic disciplines, the architecture achieves parameter-efficient expressiveness on a single consumer GPU (12GB VRAM) while maintaining graceful degradation.

Keywords: Multi-task learning, Progressive Layered Extraction, Mixture of Experts, Financial recommendation, Explainable AI, Ablation study

1 Introduction

1.1 Problem Definition

Financial product recommendation differs fundamentally from e-commerce or content recommendation. The primary deliverable is not a probability score but a *reason that the customer can accept*. Three audiences must be persuaded:

- **Customers**: "Why this product for me?" — trust leads to conversion.
- **Relationship managers**: "Why recommend this to this customer?" — sales justification.
- **Regulators** (Korean FSS [9], EU AI Act [7]): "Why was this decision made?" — compliance obligation. Korea's AI Basic Act [26] further classifies financial recommendation as potentially high-risk AI.

Existing approaches fall short on this persuasion requirement:

- **Single-task models** cannot jointly predict churn, product affinity, and customer lifetime value [3].
- **Model ensembles** (N separate models) multiply management overhead and serving cost, and "MLP #3 contributed 28%" provides no business meaning.
- **Post-hoc explanations** (SHAP, LIME) are decoupled from model internals, computationally expensive at serving time, and demonstrably unstable under input perturbation [21, 32, 33].

1.2 Core Insight

Conventional recommendation models function as "black-box shakers" — features go in, probabilities come out, with only statistical correlations as justification. As Pearl [30] argues, there is a fundamental gap between *seeing* (association) and *understanding* (causation). Humans are not persuaded by correlations; they require *causal narratives* [29].

If each expert in a multi-task model captures a *different kind of "why"* — temporal trends, hierarchical product structure, causal pathways, collaborative patterns — then the gating mechanism itself becomes an explanation: "This recommendation is driven primarily by your spending trend (Temporal, 35%) and product category fit (HGCN, 28%)."

This is the founding design principle of our architecture.

The key realization is that financial recommendation is not about a single prediction ("will this customer buy?") but about **understanding a customer as a whole person** from multiple perspectives simultaneously: Will they churn? (lifecycle). How much will they spend? (value). What product fits them? (consumption). How do they behave? (engagement). Each question demands

a different analytical lens — temporal analysis cannot answer hierarchical product fit, and graph-based collaborative signals cannot predict spending trajectories. This is why heterogeneous experts are not merely a performance optimization but a structural necessity for multi-faceted customer understanding.

This perspective aligns with a broader shift in both academia and regulation. Pearl's *Ladder of Causation* [29] distinguishes three levels: association, intervention, and counterfactuals. Most recommendation systems operate at level 1 (association); our architecture aims for levels 1–2 by incorporating structural causal reasoning. Regulatory frameworks — the EU AI Act, Korea's FSS guidelines and AI Basic Act — increasingly demand this shift toward structurally transparent explanations (detailed regulatory analysis in Section 2.3).

1.3 Contributions

1. **Heterogeneous Shared Expert Basket with Structural Collapse Guarantee**: We replace PLE's homogeneous MLP experts with seven architecturally distinct experts (DeepFM, Mamba+LNN+Transformer, HGCN, PersLay, NOTEARS, LightGCN, Optimal Transport). Unlike prior "heterogeneous" MoE work that varies expert *size* [39] or *modality* [1], we vary the fundamental *inductive bias*, providing a structural guarantee against expert collapse — a persistent failure mode in homogeneous MoE/PLE deployments [18].

2. **Inherent Explainability**: Because each expert encodes a named mathematical operation (not a generic MLP), CGC gate weights directly yield business-interpretable explanations without post-hoc attribution methods.

3. **Multi-disciplinary Feature Engineering**: Features derived from eleven academic disciplines — including unconventional applications of chemical kinetics (spending activation rate), epidemic modeling (product adoption diffusion), criminological Routine Activity Theory (transaction regularity), and wave interference (spending periodicity) — serve dual roles as learning signals and explanation material.

4. **Financial DNA Task Grouping**: Four task groups (engagement, lifecycle, value, consumption) with differentiated adaTT intra/inter transfer strengths and logit transfer for natural experience propagation.

5. **Comprehensive Ablation**: 54 scenarios (feature group / expert / task×structure) on a reproducible 1M-customer benchmark with Gaussian Copula + latent variable variance budget.

6. **Config-driven Pipeline**: End-to-end system (feature engineering → training → distillation → serving) controlled by two YAML files, enabling deployment by teams with 1–2 ML engineers.

7. **Reproducible Benchmark**: Synthetic data generation with variance budget [28] for controlled AUC ceilings, validated against XGBoost baselines, with open-source code and fixed seeds.

2 Related Work

2.1 Multi-Task Learning for Recommendation

The progression from Shared-Bottom [3] to MMoE [22] to PLE [37] reflects increasing sophistication in managing negative transfer between tasks. MMoE introduced per-task gating over a shared expert pool, while PLE further separated shared and task-specific experts with a progressive extraction structure. AdaTT [19] extended this with adaptive inter-task transfer.

Other notable MTL architectures include ESMM [23] (entire-space modeling for conversion rate), STAR [35] (star topology for multi-domain CTR), and M3oE [40] (multi-domain multi-task MoE with AutoML structure search).

However, all prior MoE/PLE architectures employ **homogeneous experts** — multiple MLPs with identical architecture but different initializations. This limits the diversity of learned representations to what parameter variation alone can achieve, and leads to a well-documented failure mode: **expert collapse**.

2.2 Mixture of Experts and the Expert Collapse Problem

The MoE paradigm [34] and its successors (Switch Transformer [8]) demonstrate the power of conditional computation. However, homogeneous MoE architectures suffer from **expert collapse**, where all gates converge to routing inputs to a single expert, effectively reducing the MoE to a shared-bottom architecture. Pinterest Engineering reported that their MMoE deployment "would oftentimes collapse into a state where all tasks use the same single expert." Kuaishou's HoME [18] confirmed that "expert collapse occurs when all gates assigned larger weights to a single shared expert and almost ignored other shared experts."

Existing mitigations operate post-hoc: Gram-Schmidt orthogonalization to force diverse expert representations, expert normalization to align output distributions, and load-balancing losses to distribute routing. These are engineering patches on a structural problem — homo-

geneous experts have no inherent reason to specialize differently because their only source of diversity is random initialization.

Recent work has explored limited forms of heterogeneity: Jamba [1] combines Transformer and Mamba layers within MoE for LLMs (2 architecture types), and MOWST [39] pairs lightweight MLP "weak experts" with GNN "strong experts" in a hierarchical arrangement (2 types, unequal roles). M3oE [40] separates experts by domain but uses identical MLP architectures within each domain. MoE++ introduces computation-level heterogeneity (zero-compute vs. FFN experts) but not architectural heterogeneity.

Gap: No prior work composes experts with **fundamentally different inductive biases** — graph convolution, state-space models, topological persistence, causal DAG constraints, optimal transport — as equal peers within a shared expert basket. Our work provides a **structural guarantee** against expert collapse: a DeepFM expert cannot converge to the same function as an HGCN expert regardless of training dynamics, because their architectures encode fundamentally different mathematical operations.

2.3 Explainability in Recommendation

SHAP [21] and LIME [32] provide model-agnostic explanations but suffer from instability, computational cost, and disconnect from model internals. Integrated Gradients [36] offers theoretically grounded attribution but still operates post-hoc.

Gap: Structural (inherent) explainability that produces business-meaningful explanations as a natural byproduct of the forward pass, without additional computation.

2.4 Financial Recommendation and Regulation

Recent work on deep learning for financial product recommendation [5, 25] demonstrates improvements over collaborative filtering, while Martinez-Plumed et al. [24] show that sequential models on longitudinal transaction data improve cross-selling prediction. However, none of these systems address regulatory explainability requirements.

The EU AI Act [7] classifies financial recommendation as high-risk AI, mandating transparency (Art. 13), human oversight (Art. 14), and robustness (Art. 15). Korea's FSS guidelines [9] and AI Basic Act [26] impose similar requirements. Current approaches rely on post-hoc SHAP/LIME for regulatory compliance, which has documented limitations in financial contexts [33].

Gap: No recommendation system provides verifiable mapping from regulatory requirements to architecture components, with explanations generated structurally rather than post-hoc.

3 Architecture

3.1 Reductionist Framework: Two Axes of Decomposition

Before describing the architecture, we present the analytical framework that governs all subsequent design decisions.

The complexity of financial customer understanding is decomposed along two orthogonal axes:

Axis 1: Who is the customer? (Financial DNA)

We decompose the question "who is this customer?" into four irreducible dimensions, each capturing a fundamentally different aspect of customer identity:

DNA	Question	Tasks
Engagement	What do they *do*?	has_nba, engagement, next_mcc, mcc_trend, top_mcc_shift
Lifecycle	Where *are* they?	churn, tenure_stage, segment
Value	How much are they *worth*?	income_tier, spend_level, cross_sell, stability
Consumption	What *will* they buy?	will_acquire_* (5), nba_primary

Table 1: Axis 1: Financial DNA decomposition. Four irreducible dimensions of customer identity.

Axis 2: What form does the information take? (Data Modality)

Customer data exists in structurally distinct modalities, each requiring a different mathematical tool to extract meaningful signals:

Modality	Nature	Optimal Tool
State	Binary/categorical, time-invariant	Feature crosses (DeepFM)
Snapshot	Numeric at a point in time	Clustering (GMM)
Short-term series	Recent sequential patterns	Attention (Transformer)
Long-term series	Multi-month trends	State space (Mamba)
Disrupted series	Irregular / dormant gaps	Adaptive ODE (LNN)
Hierarchy	Tree-structured categories	Hyperbolic embedding (HGCN)
Relations	Customer-product graph	Graph convolution (LightGCN)
Topology	Shape of behavioral patterns	Persistent homology (PersLay)
Causality	Directional dependencies	DAG constraint (NOTEARS)

Table 2: Axis 2: Data modality decomposition. Each modality demands a structurally different expert.

The cross-product of these two axes defines the architecture: each (DNA × Modality) cell determines which expert processes which features for which task group. The 7 heterogeneous experts, 12 feature groups, and 4 task groups are not arbitrary design choices but necessary consequences of this two-axis decomposition. A homogeneous MLP expert basket ignores Axis 2 entirely — it treats hierarchical, temporal, and topological data identically — which is why it cannot achieve the task-type-specific specialization that our ablation demonstrates.

3.2 Design Philosophy

The architecture emerged from severe real-world constraints at a Korean public financial institution. A team of three — one data scientist serving as PM, with financial risk management (FRM) certification and a career spanning credit/market risk analysis, regulatory compliance, MyData licensing, big data platform construction and operations, data science projects, and recommendation system management, plus two engineers — needed to replace a legacy ALS-based collaborative filtering system with a next-generation recommendation model.

The constraints were formidable: no dedicated ML infrastructure budget, a single consumer-grade GPU (NVIDIA RTX 4070, 12GB VRAM) as the only training hardware, no GPU inference servers for deployment, and strict regulatory requirements (Korean FSS AI guidelines, EU AI Act).

Rather than treating these constraints as limitations, the team adapted its methodology at every level: (1) AI-augmented development using Claude (Anthropic), Gemini (Google), and Cursor, with each team member leading a parallel team of AI agents; (2) parameter-efficient architecture design where structural inductive biases replace the brute-force capacity of large MLPs; (3) knowledge distillation to LGBM for GPU-free CPU inference on AWS Lambda; (4) config-driven pipeline requiring only two YAML files to control the entire system, enabling operation by a minimal team.

Early exploration considered a Black-Litterman-inspired approach, treating multiple models' predictions as "expert views" combined via Bayesian updating. This was abandoned because the Bayesian combination process obscured each model's contribution, making business-meaningful explanation impossible — a critical failure for regulatory compliance.

The key reframing was: instead of combining *models* externally, combine *experts* internally within a single model. This led to the selection of PLE [37] with a critical modification: replacing homogeneous MLP experts with structurally heterogeneous experts, each encoding a different inductive bias.

Four principles guide the architecture:

1. **Robust Explainability**: Model structure itself generates explanations. Each expert's name carries business meaning (e.g., "Temporal" = time patterns, "HGCN" = product hierarchy), so CGC gate weights are directly interpretable without post-hoc methods.
2. **Graceful Degradation**: Removing any single expert does not cause catastrophic performance loss. The remaining experts redistribute gate weights to compensate.
3. **Flexible Extensibility**: New features, tasks, or experts are added via YAML configuration, not code changes. The system has been extended from 4 to 18 tasks without architectural modification.
4. **Unified Manageability**: The entire pipeline (feature engineering → training → distillation → serving → monitoring) is controlled by two configuration files (`pipeline.yaml` and `feature_groups.yaml`), reducing operational overhead for teams with limited ML engineering resources.

3.3 Data Axis Classification

Financial customer data exhibits inherently multi-modal structure. We classify data along multiple axes, each mapped to an optimal feature generator and expert:

Data Axis	**Examples**	**Expert**	**Generator**
State	is_active, holdings	DeepFM	base
Snapshot	balance, demographics	GMM, DeepFM	GMM clustering
Short-term series	recent transactions	Transformer	temporal
Long-term series	monthly trends	Mamba	mamba temporal
Disrupted series	dormant→active	LNN	model derived
Hierarchy	product category tree	HGCN	product hierarchy
Relations	customer-product graph	LightGCN	graph collab.
Topology	behavioral shape	PersLay	TDA global/local
Causality	behavioral causation	Causal	causal features

Table 3: Data axis → Expert → Feature generator mapping. Each axis captures a structurally different aspect of customer behavior. Note: Short-term, Long-term, and Disrupted series map to the three sub-components of the Temporal Ensemble expert (Transformer, Mamba, LNN respectively). GMM is a feature *generator* that produces soft-clustering inputs; it is not a standalone expert.

3.4 Heterogeneous Expert Basket

Architecture diagram placeholder
Input (318 features) → 12 Feature Groups → CGC Gate → 7 Experts → 4 Task Groups → 18 Towers → Output
1. adaTT inter/intra transfer

Figure 1: Heterogeneous Expert PLE architecture overview.

Unlike standard PLE where shared experts are identical MLPs, our shared expert basket contains seven structurally distinct networks. Each expert corresponds to one or more modality axes (Table 2), ensuring that every structurally distinct data type is processed by an architecture designed for it:

Expert	**Inductive Bias**	**Captures**
DeepFM [12]	Feature interaction	2nd-order cross features
Temporal Ensemble	Multi-scale temporal	Short/long/disrupted series
HGCN [4]	Hyperbolic hierarchy	Product category tree
PersLay [2]	Topological persistence	Behavioral shape patterns
LightGCN [14]	Graph convolution	Collaborative filtering
Causal [41]	DAG constraint	Causal direction between features
Optimal Transport [6]	Distribution matching	Segment distribution shifts

Table 4: Seven heterogeneous experts with distinct inductive biases.

Because each expert has a named inductive bias, the gate weights tell the customer *why* in business terms — not "hidden unit 47 activated" but "your spending trend drove this recommendation." Each expert was selected based on a specific gap in financial customer understanding that no other expert type addresses:

- **DeepFM** [12]: Financial behavior is driven by feature interactions (e.g., income × product holdings × channel preference). FM's low-rank factorization computes 2nd-order crosses in $O(nk)$ vs. $O(n^2)$ for brute-force enumeration, while the Deep component captures higher-order interactions.
- **HGCN** [4]: Financial product catalogs (MCC codes: 10 L1 / 30 L2 / 109 leaf) are inherently tree-structured. Hyperbolic space (Poincaré ball, 8D) embeds trees with exponentially less distortion than Euclidean space [27] — critical for a 550K-node merchant hierarchy.
- **PersLay** [2]: Topological Data Analysis captures *shape* features (connected components H_0, cycles H_1, voids H_2) of customer spending patterns. H_1 loops reveal consumption cycles; H_2 voids detect systematic spending avoidance. These features are provably stable under noise (Stability Theorem).
- **Temporal Ensemble**: Financial time series combine regular snapshots, bursty transactions, and multi-month dormancy. Mamba [11] handles long-range trends in $O(n)$; LNN [13] adapts its time constants to irregular intervals; Transformer captures short-range attention patterns.
- **Causal** [41]: The NOTEARS continuous DAG constraint ($\mathrm{tr}(e^{W \circ W}) - d = 0$) learns causal direction between features, enabling "A causes B" explanations rather than "A correlates with B" [30].
- **LightGCN** [14]: Collaborative filtering via neighborhood aggregation on the customer-product bipartite graph. Stripped to essentials (no feature transform, no activation), which outperforms more complex GCN variants for recommendation.
- **Optimal Transport** [6]: Sinkhorn-regularized Wasserstein distance measures distributional shift between customer spending profiles and segment prototypes, respecting the metric structure of the feature space (unlike KL divergence).

The rationale for heterogeneous experts is also rooted in a hardware constraint: with a single consumer GPU (12GB VRAM), we cannot scale a homogeneous MLP expert to sufficient width/depth for high expressiveness. Instead, each expert leverages a *structural inductive bias* to capture patterns that would require orders of magnitude more MLP parameters. For example, HGCN embeds a product hierarchy tree in 32 hyperbolic dimensions — achieving in $O(d)$ parameters what Euclidean embeddings require $O(2^d)$ dimensions to represent without distortion [4].

The total parameter count across all seven experts remains comparable to a single large MLP, but the diversity of learned representations is fundamentally richer.

3.4.1 Temporal Ensemble: Expert-within-Expert

Financial time series exhibit a uniquely complex structure rarely seen in other domains. A single customer may simultaneously have: monthly balance snapshots (regular, long-range), daily transaction sequences (irregular, bursty), and multi-month dormancy gaps (disrupted). No single temporal model handles all three well:

- **Mamba** [11] (State Space Model): Captures long-range dependencies with $O(n)$ efficiency via selective state spaces. Ideal for monthly/quarterly trends spanning 12+ months.
- **LNN** [13] (Liquid Neural Network): Adaptive time constants that naturally handle irregular sampling intervals and dormancy gaps. When a customer is inactive for 3 months then suddenly active, LNN's continuous-time dynamics adapt without requiring imputation.
- **Transformer**: Attention-based short-range context extraction. Captures patterns within the most recent 30–90 days of transaction sequences where positional relationships matter.

The three models' outputs are concatenated and projected, mirroring the heterogeneous expert philosophy at a finer granularity. This design is validated in the ablation study (Section 5.4), where removing any single temporal component degrades performance on different task groups.

3.5 Multi-disciplinary Feature Engineering

A distinguishing aspect of this work is the systematic application of methodologies from diverse academic disciplines to financial customer behavior. Now that the expert basket has been introduced, the rationale becomes clear: each discipline's features are designed to feed a *specific expert* whose inductive bias can best exploit that signal type. Rather than relying solely on standard statistical features (mean, variance, trend), we apply domain-specific mathematical tools that each extract a structurally different signal from the same underlying data.

Discipline	Method	Dim	Financial Interpretation
Topology	Persistent Homology (TDA)	32	Behavioral shape persistence: which consumption patterns are transient vs. structural
Hyperbolic Geometry	HGCN embedding	34	Product category distance: preserving hierarchy in low dimensions
Control Theory	Mamba (State Space)	50	Long-range behavioral dependencies: how past habits influence present
Stochastic Processes	HMM state transitions	25	Latent lifecycle stages: dormant → growing → mature → at-risk
Chemical Kinetics	Reaction rate modeling	6	Spending activation rate, half-life, dormancy reactivation catalysis
Epidemiology	SIR compartment model	5	Product adoption as "infection": susceptible → adopted → churned
Criminology	Routine Activity Theory	5	Transaction regularity: burstiness, circadian variance, routine breakpoints
Signal Processing	FFT + Hilbert transform	8	Spending periodicity: spectral entropy, harmonic power, phase locking
Statistics	GMM soft membership	80	Behavioral distinctiveness
Economics	Permanent Income	3	Income stability: permanent vs. transitory

Table 3. Multi-disciplinary feature engineering in academic disciplines applied to financial behavior. Total 269 generated features + 49 base features = 318. Base features include 47 raw columns plus 2 derived columns

Several of these applications are, to our knowledge, novel in financial recommendation:

Chemical kinetics models spending behavior as a reaction system: the "activation energy" represents the threshold for a dormant customer to resume spending, while "catalysts" (e.g., payroll deposits) lower this threshold. The half-life of spending intensity after a peak captures decay dynamics that simple moving averages cannot express.

SIR epidemic model treats product adoption as a contagion process: customers are "susceptible" (not yet adopted), "infected" (recently adopted), or "recovered" (churned). The basic reproduction number R_0 of a product category across a customer's social graph predicts adoption velocity.

Friedman's Permanent Income Hypothesis [10] decomposes observed income into permanent (stable, long-term) and transitory (bonus, irregular) components using HP filter or Kalman filter. This distinction is critical for financial recommendation: a customer with high transitory income should not be recommended long-term investment products that assume stable cash flow. The FD-TVS scoring system (detailed in companion paper) uses this decomposition to weight recommendations by income stability type.

These features serve a dual purpose beyond predictive contribution: they provide *explanation vocabulary* that no standard feature can offer. "Your spending activation energy has decreased" (chemical kinetics) or "Your product adoption follows a growing epidemic curve" (SIR) are explanations grounded in established scientific frameworks.

3.6 Financial DNA Task Grouping

We organize 18 prediction tasks into four groups based on financial customer DNA. Each task group corresponds to one DNA axis (Table 1); within each group, different modality experts (Table 2) contribute differently, and the CGC gate learns the optimal mixture per task:

Group	Financial DNA	Tasks
Engagement	What does the customer do?	has_nba, engagement_score, next_mcc, top_mcc_shift, mcc_diversity_trend
Lifecycle	Where is the customer?	churn_signal, tenure_stage, segment_prediction
Value	How valuable is the customer?	income_tier, spend_level, cross_sell_count, product_stability
Consumption	What will the customer buy?	will_acquire_* (5), nba_primary

Table 6: Four task groups based on financial customer DNA.

adaTT enforces differentiated transfer: strong intra-group transfer (same DNA perspective) and weaker inter-group transfer (different perspectives, minimizing negative transfer).

Loss-level transfer. A notable departure from the original adaTT [19], which transfers at the representation level, our implementation operates at the *loss level*:

$$\mathcal{L}_i^{\text{adaTT}} = \mathcal{L}_i + \lambda \sum_{j \neq i} w_{i \to j} \cdot \mathcal{L}_j$$

The base task losses $\mathcal{L}_i$ are weighted by learned uncertainty [17], and binary classification tasks with severe class imbalance (e.g., has_nba at 3% positive rate) use focal loss [20] with task-specific α and γ parameters.

where $w_{i \to j}$ is the transfer weight from task j to task i, computed via gradient cosine similarity between task loss gradients. This design choice was motivated by two considerations: (1) representation-level transfer requires matching hidden dimensions across heterogeneous experts, which is architecturally cumbersome when experts produce outputs of different shapes and semantics; (2) loss-level transfer naturally respects the task group structure — if two tasks have similar gradient directions (high cosine similarity), their losses reinforce each other, regardless of the expert that produced the prediction.

3.7 Logit Transfer

While adaTT handles symmetric inter-task relationships, logit transfer captures *directional* dependencies that reflect the natural sequence of customer experience:

Source → Target	Method	Customer Experience	Causal Direction
has_nba → nba_primary	output_concat	Purchase decision → product selection	Sequential
engagement_score → has_nba	hidden_concat	Activity level → purchase probability	Leading indicator
spend_level → will_acquire_*	residual	Spending capacity → category intent	Enabling factor
churn → has_nba	output_concat	Retention risk → acquisition opportunity	Inverse correlation

Table 7: Logit transfer relationships reflecting natural customer experience flow.

These transfers connect DNA groups (Table 1): engagement→engagement (intra-group), value→consumption (inter-group), reflecting the natural sequence of customer experience. The transfer directions are not learned from data but specified based on domain knowledge of the customer journey. This is a deliberate design choice: while the *strength* of transfer is learned (via adaTT affinity), the *direction* is fixed by business logic, providing an additional layer of interpretability.

3.8 Inherent Explainability

A central claim of this work is that the heterogeneous expert structure provides *inherent* explainability — explanations that *persuade* customers, relationship managers, and regulators emerge as a natural byproduct of the forward pass, not as a separate post-hoc computation.

Mechanism. The CGC (Customized Gate Control) module computes a softmax attention over the K experts for each task t:

$$w_t = \text{softmax}(W_t \cdot h + b_t) \in \mathbb{R}^K$$

where h is the input representation. The weight $w_{t,k}$ directly indicates how much expert k contributed to task t's prediction.

Business interpretability. Because each expert k has a named inductive bias (e.g., k = "Temporal" means time-series patterns, k = "HGCN" means product hierarchy),

$w_{t,k}$ carries business meaning without additional interpretation:

> *Example*: For customer c, the recommendation of investment funds is driven by:
> Temporal (0.35) — spending has been increasing over 3 months
> HGCN (0.28) — current products are structurally close to investment category
> DeepFM (0.22) — income × product-holding interaction pattern
> Others (0.15)

Comparison with post-hoc methods. SHAP and LIME operate at the *feature* level ("feature_237 contributed 0.12"), requiring an additional mapping step to translate feature attributions into business narratives. Moreover, LIME's local linear approximation is demonstrably unstable — small input perturbations can dramatically change the explanation [32]. In contrast, gate weights are deterministic functions of the input and change smoothly with input variation.

Dual role of features. An important design insight is that features serve two distinct purposes: (1) predictive contribution to AUC, and (2) explanation material for recommendation reasons. Even features with marginal predictive contribution (e.g., TDA topological features may add only $\Delta\text{AUC} \approx 0.01$) provide irreplaceable explanation vocabulary. "Your spending pattern shows a *persistently stable shape*" cannot be generated from any other feature type. This dual role motivates our multi-disciplinary feature engineering and is validated in the ablation study.

4 Training Pipeline

4.1 Config-Driven Design

The entire pipeline is controlled by two YAML files: `pipeline.yaml` (model, training, deployment settings) and `feature_groups.yaml` (feature generation, expert routing). Adding a new dataset, task, or expert requires only configuration changes.

4.2 Data Processing

Following the principle of *pandas-free* data engineering for scalability, the pipeline uses a tiered backend strategy: cuDF (GPU columnar) for generator fitting/transformation, DuckDB [31] (CPU columnar with disk spill) for SQL-based aggregation and I/O, and PyArrow for zero-copy parquet loading into training tensors. pandas is used only as a last-resort fallback for datasets under 10K rows.

This backend choice has practical implications: DuckDB handles the 1M-row dataset with a 4GB memory limit and automatically spills to disk, enabling processing on commodity hardware. cuDF accelerates generator fitting (GMM, HMM, Mamba) on GPU, reducing Phase 0 feature engineering from 30 minutes (pandas) to 8 minutes.

4.3 Data Leakage Prevention

Data leakage is a pervasive risk in multi-stage pipelines where features and labels are processed in separate stages. We implement three structural safeguards:

1. **Generator label exclusion**: Feature generators (GMM, model-derived, etc.) automatically exclude all label columns from their input. Without this guard, we observed XGBoost AUC of 1.0 on the generated features — the generator had encoded label information directly into features.
2. **Temporal split with gap**: A configurable `gap_days` parameter ensures no temporal overlap between training and validation windows. For cross-sectional (single-snapshot) data, the system auto-detects and falls back to random splitting.
3. **Scaler train-only fitting**: StandardScaler is fit exclusively on the training split; validation and test data are transformed using training statistics.
4. **LeakageValidator**: A pre-training check computes Pearson correlation between every feature and every label (on a 50K subsample for efficiency). Correlations above 0.95 trigger a warning and investigation.

5 Experiments

5.1 Benchmark Data

We construct a 1M-customer synthetic benchmark using a four-layer generative model, inspired by the Synthetic Data Vault framework [28] and conditional generative approaches [38], but with a novel *variance budget* mechanism for controllable difficulty:

1. **Latent Personas**: 6 GMM-fitted personas with 5D continuous latent vector (70% persona-conditioned, 30% independent noise).
2. **Gaussian Copula Demographics**: Correlated demographic variables preserving realistic joint distributions [28].
3. **Vectorized Transactions**: Per-customer transaction sequences as LIST columns.
4. **Variance Budget Labels**: Each label's predictability is controlled via $\text{logit} = \sqrt{f_{\text{obs}}} \cdot z_{\text{obs}} + \sqrt{f_{\text{lat}}} \cdot z_{\text{lat}} + \sqrt{f_{\text{noise}}} \cdot \varepsilon$, with post-hoc label noise flipping.

Tier	Labels	f_{obs}	f_{noise}	XGB AUC
Easy	segment, income_tier	determ.	–	0.95–1.0
Core	has_nba, churn_signal	0.04	0.68	0.58–0.65
Hard	will_acquire_*	0.03	0.72	0.50–0.56
V.Hard	top_mcc, top_mcc_shift	0.02	0.78	0.50–0.51

Table 8: Variance budget per label tier. XGB AUC ceiling validates difficulty control.

5.2 Experimental Setup

The ablation validates whether each expert provides a distinct "why" for different task types — confirming that heterogeneous experts are not merely a performance trick but a structural requirement for multi-faceted persuasion.

- **Data**: 1M customers, 318 features, 18 tasks.
- **Hardware**: NVIDIA RTX 4070 (12GB) local; AWS g5.xlarge (A10G 24GB) cloud.
- **Training**: 5+5 epochs (phase1 + phase2), batch 6144, lr 0.008, AMP, early stopping patience 3.
- **Metrics**: AUC (binary), F1 macro (classification), MAE/R^2 (regression).

5.3 Feature Group Ablation (RQ1)

Scenario	Features	Avg AUC	Δ vs Full
full	318	–	baseline
base_only	49	–	–
base+tda	65	–	–
base+hmm	74	–	–
base+mamba	99	–	–
full−tda	300	–	–
full−hmm	291	–	–
...	...	...	...

Table 9: Feature group ablation results. TODO: fill after ablation.

5.4 Expert Ablation (RQ2)

5.5 Task × Structure Cross Ablation (RQ3)

5.6 Graceful Degradation (RQ4)

5.7 Explainability Analysis (RQ5)

6 Discussion

6.1 Findings Summary

6.2 Practical Implications

Resource-constrained development. This system was built without dedicated ML infrastructure budget, on a single consumer GPU, by a three-person team augmented with AI development agents. This demonstrates that complex multi-task recommendation systems are no longer exclusive to organizations with large ML teams and GPU clusters. The key enablers were: (1) config-driven architecture minimizing code changes, (2) AI agents handling parallel implementation tasks under human architectural guidance, (3) heterogeneous expert design achieving expressiveness through structural bias rather than parameter scale, and (4) knowledge distillation eliminating GPU serving costs.

Feature engineering philosophy. A deeper lesson from this work is that *what to observe* matters more than *how to model*. The heterogeneous expert architecture is effective not because of architectural novelty alone, but because each expert is paired with features that ask a **specific question** about the customer: "Is their spending decaying with a half-life?" (chemical kinetics), "Is product adoption spreading like contagion?" (SIR epidemiology), "Is their income permanent or transitory?" (Friedman PIH [10]). The model merely combines answers to these questions. If the questions are shallow — mean, variance, trend — even the most sophisticated architecture produces shallow understanding. This suggests that multi-disciplinary feature engineering, often treated as preprocessing, deserves elevation to a first-class architectural design decision, co-designed with the expert types that will consume each feature group.

Generalizability beyond finance. While this work targets financial product recommendation, the underlying principle — understanding a single user from multiple perspectives simultaneously via heterogeneous experts — applies to any domain where personalization requires multi-faceted user modeling. Healthcare (diagnosis + treatment + risk + lifestyle), education (knowledge level + learning style + engagement + career path), and insurance (risk assessment + product fit + claim predic-

tion + retention) all face structurally similar challenges: multiple interdependent prediction tasks about the same individual, where different analytical lenses capture different aspects. We expect the heterogeneous expert PLE pattern to generalize to these domains, with domain-specific expert types replacing the financial experts while retaining the structural benefits of collapse resistance and inherent explainability.

Infrastructure choice. Financial institutions differ fundamentally from big tech in ML infrastructure needs. Big tech operates hundreds of GPUs across dozens of teams with hourly retraining cycles, justifying Kubernetes-based GPU schedulers (Kubeflow). Financial institutions typically train models daily to weekly, with fewer than 10 models in production, making dedicated GPU clusters an over-investment. Our SageMaker + Lambda serverless architecture matches this reality: GPU resources are provisioned on-demand for training (Spot instances at 60–70% discount) and released immediately after, while serving runs on CPU-only Lambda functions via knowledge distillation to LGBM (detailed in companion paper).

Operational simplicity. The config-driven design (two YAML files control the entire pipeline) enables teams with 1–2 ML engineers to operate the system. Adding a new product category, task, or feature group requires only configuration changes, not code modifications. This stands in contrast to model ensemble approaches where each model is a separate management point with its own training pipeline, serving endpoint, and monitoring dashboard.

6.3 Limitations

- **Synthetic benchmark**: While our variance-budget approach controls difficulty levels, synthetic data cannot fully capture the distributional complexity of real financial data. We plan to validate on production data from a partner institution in future work.
- **Single-GPU training**: The current implementation runs on a single GPU. DDP (DistributedDataParallel) support is architecturally designed but not yet experimentally validated. For the ablation scenarios studied here, single-GPU training completes within acceptable time.
- **Phase 2 numerical stability**: Mixed-precision (FP16) training in Phase 2 requires careful handling of log-domain computations (CGC entropy, OT Sinkhorn) to avoid FP16 underflow. We document four specific numerical fixes that were necessary for stable Phase 2 training.
- **LLM dependency**: Recommendation reason generation relies on LLM inference, introducing latency and cost trade-offs (detailed in companion paper).

7 Conclusion

We presented Heterogeneous Expert PLE, a multi-task learning architecture for financial product recommendation that addresses the fundamental requirement of *persuading humans* — not merely predicting their behavior.

By composing seven structurally distinct experts within PLE's shared basket, we achieve three properties simultaneously: (1) parameter-efficient expressiveness under hardware constraints, (2) inherent explainability through business-interpretable gate weights, and (3) graceful degradation validated through comprehensive ablation.

The key insight is that explanation quality depends not on post-hoc attribution methods but on the model's internal structure. The two-axis decomposition (Financial DNA × Data Modality) ensures that every design decision — which experts to build, which features to engineer, which tasks to group — traces back to a principled analytical framework. When each expert carries a named inductive bias — temporal dynamics, product hierarchy, causal pathways — the gating mechanism *is* the explanation.

The architecture, benchmark data, and ablation framework are released as open source to enable reproduction and extension by the research community. A companion paper addresses the downstream pipeline: knowledge distillation [15] to LGBM [16], multi-agent recommendation reason generation, and regulatory compliance mapping for Korean FSS and EU AI Act requirements.

Acknowledgments

The code implementation and manuscript drafting were assisted by Claude (Anthropic) as an AI coding and writing tool. The architectural decisions, domain knowledge, experimental design, and research direction were led by the human authors.

Bibliography

[1] AI21 Labs. 2024. Jamba: A Hybrid Transformer-Mamba Language Model. *arXiv preprint arXiv:2403.19887* (2024).

[2] Mathieu Carrière, Frédéric Chazal, Yuichi Ike, Théo Lacombe, Martin Royer, and Yuhei Umeda. 2020. PersLay: A Neural Network Layer for Persistence Diagrams and New Graph Topological Signatures. *Journal of Machine Learning Research* (2020).

[3] Rich Caruana. 1997. Multitask Learning. *Machine Learning* 28, 1 (1997), 41–75.

[4] Ines Chami, Rex Ying, Christopher Ré, and Jure Leskovec. 2019. Hyperbolic Graph Convolutional Neural Networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[5] Wei Chen, Yong Liu, and Hao Zhang. 2024. Deep Learning-Powered Financial Product Recommendation System in Banks: Integration of Transformer and Transfer Learning. *Journal of Organizational and End User Computing* 36, 1 (2024).

[6] Marco Cuturi. 2013. Sinkhorn Distances: Lightspeed Computation of Optimal Transport. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2013.

[7] European Parliament and Council. 2024. Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence (AI Act).

[8] William Fedus, Barret Zoph, and Noam Shazeer. 2022. Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity. *Journal of Machine Learning Research* (2022).

[9] Financial Services Commission (Korea). 2024. Financial Sector AI Guidelines.

[10] Milton Friedman. 1957. *A Theory of the Consumption Function*. Princeton University Press.

[11] Albert Gu and Tri Dao. 2024. Mamba: Linear-Time Sequence Modeling with Selective State Spaces. (2024).

[12] Huifeng Guo, Ruiming Tang, Yunming Ye, Zhenguo Li, and Xiuqiang He. 2017. DeepFM: A Factorization-Machine based Neural Network for CTR Prediction. In *Proceedings of the 26th International Joint Conference on Artificial Intelligence (IJCAI)*, 2017.

[13] Ramin Hasani, Mathias Lechner, Alexander Amini, and others. 2021. Liquid Time-constant Networks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2021.

[14] Xiangnan He, Kuan Deng, Xiang Wang, Yan Li, Yongdong Zhang, and Meng Wang. 2020. LightGCN: Simplifying and Powering Graph Convolution Network for Recommendation. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2020.

[15] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. 2015. Distilling the Knowledge in a Neural Network. In *NIPS Deep Learning and Representation Learning Workshop*, 2015.

[16] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. 2017. LightGBM: A Highly Efficient Gradient Boosting Decision Tree. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[17] Alex Kendall, Yarin Gal, and Roberto Cipolla. 2018. Multi-Task Learning Using Uncertainty to Weigh Losses for Scene Geometry and Semantics. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.

[18] Jiaxin Li and others. 2024. HoME: Hierarchy of Multi-Gate Experts for Multi-Task Learning at Kuaishou. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2024.

[19] Yankai Li and others. 2023. AdaTT: Adaptive Task-to-Task Fusion Network for Multi-task Learning in Recommendations. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2023.

[20] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. 2017. Focal Loss for Dense Object Detection. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2017.

[21] Scott M Lundberg and Su-In Lee. 2017. A Unified Approach to Interpreting Model Predictions. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[22] Jiaqi Ma, Zhe Zhao, Xinyang Yi, Jilin Chen, Lichan Hong, and Ed H Chi. 2018. Modeling Task Relationships in Multi-task Learning with Multi-gate Mixture-of-Experts. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2018.

[23] Xiao Ma, Liqin Zhao, Guan Huang, Zhi Wang, Zelin Hu, Xiaoqiang Zhu, and Kun Gai. 2018. Entire Space Multi-Task Model: An Effective Approach for Estimating Post-Click Conversion Rate. In *Proceedings of the 41st International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2018.

[24] Fernando Martinez-Plumed and others. 2023. Improving the Predictive Accuracy of the Cross-selling

of Consumer Loans Using Deep Learning Networks. *Annals of Operations Research* (2023).

[25] Ali Met, Deniz Korkmaz, and Kerem Tuna. 2024. Product Recommendation System With Machine Learning Algorithms for SME Banking. *International Journal of Intelligent Systems* 2024, (2024).

[26] National Assembly of Korea. 2024. Act on the Development of Artificial Intelligence and Establishment of Trust (AI Basic Act).

[27] Maximilian Nickel and Douwe Kiela. 2017. Poincaré Embeddings for Learning Hierarchical Representations. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[28] Neha Patki, Roy Wedge, and Kalyan Veeramachaneni. 2016. The Synthetic Data Vault. In *IEEE International Conference on Data Science and Advanced Analytics (DSAA)*, 2016.

[29] Judea Pearl and Dana Mackenzie. 2018. *The Book of Why: The New Science of Cause and Effect.* Basic Books.

[30] Judea Pearl. 2009. *Causality: Models, Reasoning, and Inference* (2nd ed.). Cambridge University Press.

[31] Mark Raasveldt and Hannes Mühleisen. 2019. DuckDB: An Embeddable Analytical Database. (2019).

[32] Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. 2016. Why Should I Trust You?: Explaining the Predictions of Any Classifier. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016.

[33] Amr Salih and others. 2023. Examining the Limitations of Post-hoc Explainability Methods in Finance. *Frontiers in Artificial Intelligence* (2023).

[34] Noam Shazeer, Azalia Mirhoseini, Krzysztof Mahdavi, and others. 2017. Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer. (2017).

[35] Xiang-Rong Sheng, Liqin Zhao, Guorui Zhou, Xinyao Ding, Binding Dai, Qiang Luo, Siran Yang, Jingshan Lv, Chi Zhang, Hongbo Deng, and Xiaoqiang Zhu. 2021. STAR: One-pass Single-Arch Multi-task Architecture for Cross-Domain CTR Prediction. In *Proceedings of the 30th ACM International Conference on Information and Knowledge Management (CIKM)*, 2021.

[36] Mukund Sundararajan, Ankur Taly, and Qiqi Yan. 2017. Axiomatic Attribution for Deep Networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, 2017.

[37] Hongyan Tang, Junning Liu, Ming Zhao, and Xudong Gong. 2020. Progressive Layered Extraction (PLE): A Novel Multi-Task Learning (MTL) Model for Personalized Recommendations. In *Proceedings of the 14th ACM Conference on Recommender Systems (RecSys)*, 2020.

[38] Lei Xu, Maria Skoularidou, Alfredo Cuesta-Infante, and Kalyan Veeramachaneni. 2019. Modeling Tabular Data using Conditional GAN. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[39] Hanqing Zeng and others. 2024. Mixture of Weak and Strong Experts on Graphs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[40] Zijian Zhang and others. 2024. M3oE: Multi-Domain Multi-Task Mixture-of Experts Recommendation Framework. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2024.

[41] Xun Zheng, Bryon Aragam, Pradeep Ravikumar, and Eric P Xing. 2018. DAGs with NO TEARS: Continuous Optimization for Structure Learning. (2018).